

GPUMODE Lecture Study Notes: Lecture 30

Quantized Training

Core Problem the Lecture Addresses

This lecture explains how quantization can be used during training, not only during inference. In inference, quantization usually means storing weights or activations in lower precision so that the model uses less memory bandwidth and faster matrix multiplication kernels. Training is harder because the system must also preserve gradients, optimizer states, saved activations, and stable weight updates.

The central problem is:

Can low-bit data types reduce training memory or improve training speed without destroying convergence?

The lecture studies three main targets for quantization:

1. **Optimizer states:** store Adam or SGD momentum buffers in lower precision.
2. **Trainable weights:** keep the model weights themselves in a quantized representation.
3. **Matrix multiplications:** use INT8 or FP8 tensor cores during forward and backward passes.

The main hardware motivation is that lower precision tensor cores can be much faster than BF16 tensor cores. The main numerical problem is that low-bit formats have limited range and limited precision, so small updates may disappear.

Required Background

Training Memory Profile

A training step contains a forward pass, backward pass, and optimizer step. The total memory footprint can be approximated as

$$M_{\text{train}} = M_{\text{params}} + M_{\text{opt}} + M_{\text{act}} + M_{\text{grad}} + M_{\text{comm}}.$$

Here M_{params} is parameter memory, M_{opt} is optimizer-state memory, M_{act} is activation memory, M_{grad} is gradient memory, and M_{comm} is distributed communication memory.

For Adam, there are usually two optimizer-state tensors per parameter: the first moment m_t and the second moment v_t . If the model has N parameters and the optimizer states are FP32, Adam state memory is

$$M_{\text{Adam}} = 2N \cdot 4 \text{ bytes} = 8N \text{ bytes}.$$

For an 8 billion parameter model, this becomes

$$M_{\text{Adam}} = 8 \cdot 8 \times 10^9 = 64 \times 10^9 \text{ bytes} \approx 64 \text{ GB}.$$

This is why optimizer-state quantization is attractive.

Post-Training Quantization Recap

In post-training quantization, common strategies include:

- **Weight-only quantization:** only the weights are stored in low precision.
- **Dynamic activation quantization:** activations are quantized at runtime.
- **Static activation quantization:** activation scales are precomputed from calibration data.

For a linear layer,

$$Y = XW^T.$$

If the weight is quantized as

$$W \approx s_W Q_W,$$

then the computation becomes

$$Y \approx X(s_W Q_W)^T.$$

The dequantization can happen before matrix multiplication or inside the matrix multiplication kernel.

GPU Execution Model

A GPU executes kernels using a grid of thread blocks. Each thread block runs on a streaming multiprocessor, also called an SM. Threads are grouped into warps, and on NVIDIA GPUs a warp usually contains

32 threads.

The main memory hierarchy is:

- **Global memory:** large HBM memory with high latency and high bandwidth.
- **L2 cache:** shared across SMs.
- **Shared memory:** programmer-managed memory local to an SM.
- **Registers:** private per-thread storage with the lowest latency.

Quantized training is useful only when low-bit storage or low-bit compute reduces the actual bottleneck. If a dequantized FP32 tensor is written back to global memory, the memory-saving benefit can disappear.

Main Concepts

Low-Bit Optimizers

A low-bit optimizer stores optimizer states in low precision. For Adam, the update equations are

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (1)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (2)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad (3)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad (4)$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (5)$$

The states m_t and v_t are large. If they are stored in INT8 instead of FP32, the theoretical optimizer-state memory ratio is

$$\frac{8}{32} = \frac{1}{4}.$$

If they are stored in 4-bit precision, the ratio is

$$\frac{4}{32} = \frac{1}{8}.$$

The important implementation pattern is:

quantized state in HBM $\rightarrow$ dequantize inside kernel $\rightarrow$ optimizer math $\rightarrow$ requantize inside kernel $\rightarrow$ quantized

The dequantized optimizer state should not be materialized in global memory. It should live in registers or shared memory inside the kernel.

A low-bit optimizer only saves memory if the full high-precision optimizer state never appears as a persistent global-memory tensor.

Why Small Quantization Groups Matter

A common quantization rule is

$$s_G = \frac{\max_{x \in G} |x|}{q_{\max}},$$

$$q_i = \text{round} \left(\frac{x_i}{s_G} \right),$$

$$x_i \approx s_G q_i.$$

Here G is a quantization group and $q_{\max}$ is the largest representable integer value.

If G is the whole tensor, computing the scale requires a tensor-wide reduction:

$$\max_i |x_i|.$$

That is difficult to fuse into a single optimizer kernel because it requires global coordination. Instead, low-bit optimizers often use small groups or blocks:

$$G_j = \{x_j, x_{j+1}, \dots, x_{j+B-1}\}.$$

Then the scale is local:

$$s_j = \frac{\max_{k \in [j, j+B)} |x_k|}{q_{\max}}.$$

This can be computed within one block or tile.

SGD with Momentum as a Simple Example

SGD with momentum keeps a buffer b_t :

$$b_t = \mu b_{t-1} + g_t, \tag{6}$$

$$\theta_t = \theta_{t-1} - \eta b_t. \tag{7}$$

A low-bit optimizer can store b_t in BF16, FP8, INT8, or another compact format. The safe pattern is to upcast for arithmetic and downcast only when writing the state back:

$$b_{t-1}^{\text{FP32}} = \text{cast}_{\text{FP32}}(b_{t-1}^{\text{low}}),$$

$$b_t^{\text{FP32}} = \mu b_{t-1}^{\text{FP32}} + g_t,$$

$$b_t^{\text{low}} = \text{quantize}(b_t^{\text{FP32}}).$$

Quantized-Weight Training

The Weight Update Underflow Problem

Quantized-weight training means the model keeps only a low-precision copy of the trainable weights. The standard update is

$$\theta_{t+1} = \theta_t - \eta u_t,$$

where u_t is the update direction produced by the optimizer.

The problem is that the update ηu_t may be too small to change a low-precision value. For example, suppose a value is stored as an integer and we repeatedly add 0.2:

$$x_0 = 10,$$

$$x_{t+1} = \text{round}(x_t + 0.2).$$

With deterministic rounding,

$$\text{round}(10.2) = 10.$$

After 100 updates, the mathematically correct value is

$$10 + 100 \cdot 0.2 = 30.$$

But deterministic rounding may keep the value stuck at 10. This is a training failure caused by low-precision update underflow.

Stochastic Rounding

Stochastic rounding fixes this by making rounding probabilistic. Let

$$a = \lfloor x \rfloor,$$

$$f = x - \lfloor x \rfloor.$$

Then stochastic rounding returns

$$\text{SR}(x) = \begin{cases} a + 1, & \text{with probability } f, \\ a, & \text{with probability } 1 - f. \end{cases}$$

The expectation is

$$\mathbb{E}[\text{SR}(x)] = (a + 1)f + a(1 - f) \tag{8}$$

$$= af + f + a - af \tag{9}$$

$$= a + f \tag{10}$$

$$= x. \tag{11}$$

So stochastic rounding is unbiased:

$$\mathbb{E}[\text{SR}(x)] = x.$$

This means small updates are not erased in expectation. Individual values are noisy, but across many parameters the update direction is preserved statistically.

BF16 Also Has This Problem

BF16 has the same exponent width as FP32 but far fewer mantissa bits. BF16 can represent large and small magnitudes, but its spacing between adjacent values can be coarse.

A small update can vanish when

$$|\Delta\theta| < \frac{1}{2} \text{ulp}(\theta),$$

where $\text{ulp}(\theta)$ is the spacing between adjacent representable BF16 values near θ .

Therefore BF16 training can also benefit from stochastic rounding, especially when no FP32 master weights are kept.

Low-Bit Matrix Multiplication During Training

Motivation

Another approach is to keep high-precision weights for optimizer updates but use low-bit matrix multiplication for the expensive GEMMs. For a linear layer,

$$Y = XW^T.$$

Quantize X and W :

$$X \approx S_X Q_X,$$

$$W \approx S_W Q_W.$$

Then

$$Y \approx S_X S_W Q_X Q_W^T.$$

If Q_X and Q_W are INT8, the product can use INT8 tensor cores.

Tensorwise Scaling

The simplest scaling strategy uses one scale per tensor:

$$A \approx s_A Q_A,$$

$$B \approx s_B Q_B.$$

Then

$$C = AB \approx s_A s_B Q_A Q_B.$$

This is simple but can be sensitive to outliers because one large value controls the scale for the whole tensor.

Rowwise and Columnwise Scaling

For matrix multiplication,

$$C = AB,$$

with

$$A \in \mathbb{R}^{M \times K},$$

$$B \in \mathbb{R}^{K \times N},$$

$$C \in \mathbb{R}^{M \times N}.$$

Each output element is

$$C_{ij} = \sum_{k=1}^K A_{ik} B_{kj}.$$

If A uses rowwise scaling,

$$A_{ik} \approx s_i^A Q_{ik}^A,$$

and B uses columnwise scaling,

$$B_{kj} \approx s_j^B Q_{kj}^B,$$

then

$$C_{ij} \approx \sum_{k=1}^K (s_i^A Q_{ik}^A) (s_j^B Q_{kj}^B) \tag{12}$$

$$= s_i^A s_j^B \sum_{k=1}^K Q_{ik}^A Q_{kj}^B. \tag{13}$$

This is efficient because the scales are outside the reduction over k . A GEMM kernel can compute the integer dot product first and then apply

$$C_{ij} \leftarrow s_i^A s_j^B C_{ij}^{\text{int}}.$$

This scaling can be fused into the GEMM epilogue.

Why Inner-Dimension Scaling Is Harder

If scaling depends on the reduction index k , then

$$A_{ik} \approx s_k^A Q_{ik}^A,$$

$$B_{kj} \approx s_k^B Q_{kj}^B.$$

The output becomes

$$C_{ij} \approx \sum_{k=1}^K s_k^A s_k^B Q_{ik}^A Q_{kj}^B.$$

Now the scale is inside the reduction loop. This is harder to implement efficiently because tensor cores expect a regular dot product.

Hardware-Level Explanation

Where Quantization Helps

Quantization helps training in two different ways:

1. **Memory reduction:** fewer bytes are read from and written to HBM.
2. **Compute acceleration:** lower precision tensor cores execute more operations per second.

For optimizer states, the main benefit is memory reduction. For INT8 matrix multiplication, the main benefit is compute acceleration.

HBM Traffic and Kernel Fusion

A bad low-bit optimizer implementation does this:

INT8 state in HBM $\rightarrow$ FP32 state in HBM $\rightarrow$ optimizer update $\rightarrow$ INT8 state in HBM.

A good implementation does this:

INT8 state in HBM $\rightarrow$ FP32 values in registers or shared memory $\rightarrow$ optimizer update $\rightarrow$ INT8 state in HBM

The second version avoids writing the full FP32 state to global memory.

Tensor Cores

Tensor cores accelerate matrix multiply-accumulate. An INT8 tensor core operation conceptually performs

$$C_{ij} = \sum_{k=1}^K A_{ik}^{\text{INT8}} B_{kj}^{\text{INT8}}.$$

The accumulation is usually wider than INT8:

$$\text{INT8} \times \text{INT8} \rightarrow \text{INT32 accumulation}.$$

The output is then scaled and converted to FP16, BF16, FP32, or another desired type.

Amdahl’s Law for Training Speedup

If a fraction f of the runtime is accelerated by a factor r , total speedup is

$$S = \frac{1}{(1 - f) + \frac{f}{r}}.$$

For example, if GEMMs are 70% of the runtime and INT8 makes GEMMs 3.5 times faster,

$$S = \frac{1}{0.3 + \frac{0.7}{3.5}} = 2.$$

So even very fast INT8 tensor cores cannot accelerate work that is not GEMM, such as normalization, activation functions, optimizer overhead, communication, and data movement.

Mathematical Reasoning

Optimizer-State Memory

Let N be the number of parameters and b be the number of bits per optimizer-state element. Adam has two state tensors, so

$$M_{\text{Adam}}(b) = 2N \cdot \frac{b}{8} \text{ bytes}.$$

For FP32,

$$M_{\text{Adam}}(32) = 2N \cdot 4 = 8N \text{ bytes.}$$

For INT8,

$$M_{\text{Adam}}(8) = 2N \cdot 1 = 2N \text{ bytes.}$$

The memory ratio is

$$\frac{M_{\text{Adam}}(8)}{M_{\text{Adam}}(32)} = \frac{2N}{8N} = \frac{1}{4}.$$

For 4-bit states,

$$M_{\text{Adam}}(4) = 2N \cdot \frac{1}{2} = N \text{ bytes.}$$

The memory ratio is

$$\frac{M_{\text{Adam}}(4)}{M_{\text{Adam}}(32)} = \frac{N}{8N} = \frac{1}{8}.$$

Variance of Stochastic Rounding

Let

$$x = a + f,$$

where $a = \lfloor x \rfloor$ and $0 \leq f < 1$.

The stochastic rounding error is

$$e = \text{SR}(x) - x.$$

If the value rounds down, then

$$e = a - (a + f) = -f.$$

If the value rounds up, then

$$e = a + 1 - (a + f) = 1 - f.$$

The variance is

$$\text{Var}(e) = (1 - f)(-f)^2 + f(1 - f)^2 \quad (14)$$

$$= (1 - f)f^2 + f(1 - 2f + f^2) \quad (15)$$

$$= f^2 - f^3 + f - 2f^2 + f^3 \quad (16)$$

$$= f - f^2 \quad (17)$$

$$= f(1 - f). \quad (18)$$

The maximum variance happens when $f = 1/2$:

$$\text{Var}(e)_{\max} = \frac{1}{4}.$$

Forward and Backward Linear Layer Equations

For a linear layer,

$$Y = XW^T.$$

Given upstream gradient

$$G_Y = \frac{\partial L}{\partial Y},$$

the input gradient is

$$G_X = G_Y W.$$

The weight gradient is

$$G_W = G_Y^T X.$$

The scaling layout that is convenient for the forward pass may not be convenient for the backward pass. For example, if W is rowwise scaled for W^T in the forward pass, then the same layout may become columnwise relative to the backward GEMM. This creates friction when trying to reuse the same quantized representation.

Code Walkthrough

SGD with Low-Precision Momentum Buffer

```
import torch

def sgd_step(param, grad, momentum_buffer, lr=1e-3, momentum=0.9):
    buf_fp32 = momentum_buffer.float()
    new_buf = momentum * buf_fp32 + grad.float()
    param.add_(new_buf, alpha=-lr)
    momentum_buffer.copy_(new_buf)
    return param, momentum_buffer
```

The important line is the copy into the momentum buffer. If the momentum buffer is BF16, this copy casts to BF16. If the momentum buffer is represented by a quantized tensor subclass, the copy can trigger quantization.

The efficient GPU implementation should fuse the upcast, update, and downcast into one kernel so that the full FP32 momentum tensor is not written to global memory.

Integer Stochastic Rounding

```
import torch

def stochastic_round_int(x):
    floor_x = torch.floor(x)
    frac = x - floor_x
    random_value = torch.rand_like(x)
    rounded = floor_x + (random_value < frac).to(x.dtype)
    return rounded
```

This implements

$$\text{SR}(x) = \begin{cases} \lfloor x \rfloor + 1, & \text{with probability } x - \lfloor x \rfloor, \\ \lfloor x \rfloor, & \text{otherwise.} \end{cases}$$

A simple experiment is:

```
x = torch.tensor(10.0)

for _ in range(100):
    x = stochastic_round_int(x + 0.2)

print(x)
```

The printed value will be random but centered near 30.

BF16 Stochastic Rounding Sketch

```
import torch

def stochastic_round_bf16(x_fp32):
    x_int = x_fp32.view(torch.int32)

    upper = x_int >> 16
    lower = x_int & 0xFFFF

    rnd = torch.randint(
        low=0,
        high=2**16,
        size=x_fp32.shape,
        device=x_fp32.device,
        dtype=torch.int32,
    )

    upper = upper + (rnd < lower).to(torch.int32)
    rounded_int = upper << 16

    return rounded_int.view(torch.float32).to(torch.bfloat16)
```

This code treats the lower 16 bits of the FP32 representation as the fractional part that would be removed when converting to BF16. The random integer decides whether to round upward.

Scaled INT8 Matrix Multiplication

```
def scaled_int8_matmul(q_a, q_b, row_scale, col_scale):
    acc = torch.matmul(q_a.to(torch.int32), q_b.to(torch.int32))

    out = acc.float()
    out = out * row_scale[:, None]
    out = out * col_scale[None, :]

    return out
```

A production Triton kernel should not do this as separate PyTorch operations. Instead, it should fuse the scale multiplication into the GEMM epilogue:

$$C_{ij} = s_i^A s_j^B C_{ij}^{\text{int}}.$$

This avoids extra global-memory reads and writes.

Performance Intuition

Why Low-Bit Optimizers Mainly Save Memory

Optimizer-state quantization reduces persistent state memory. It may also reduce optimizer-step bandwidth. But it does not directly accelerate the main transformer GEMMs. Therefore, it helps most when optimizer memory or optimizer bandwidth is a bottleneck.

Why INT8 Matmul Can Be Very Fast

INT8 tensor cores can have much higher throughput than BF16 tensor cores. If GEMM dominates runtime, INT8 training can significantly improve end-to-end speed.

However, the speedup depends on:

- how much runtime is actually spent in GEMMs,
- whether quantization overhead is fused,
- whether scales are loaded efficiently,
- whether matrix sizes are tensor-core friendly,
- whether dynamic shapes cause recompilation,
- whether non-GEMM operations dominate.

Why Rowwise Scaling Is More Accurate Than Tensorwise Scaling

Tensorwise scaling uses

$$s = \frac{\max |X|}{127}.$$

One outlier can make s too large, wasting many INT8 levels for normal values.

Rowwise scaling uses

$$s_i = \frac{\max_j |X_{ij}|}{127}.$$

This adapts the scale to each row and often reduces quantization error.

Why Quantized-Weight Training Is Tricky

Quantized-weight training has several issues:

- Small updates can vanish without stochastic rounding.

- Scale computation can require reductions.
- The quantized layout used in forward may not match backward.
- Tensor subclasses can introduce surprising behavior if operations such as copy perform stochastic rounding.
- The expected memory savings may not appear if intermediate high-precision values are materialized.

BitNet and Very Low-Bit Training

BitNet-style training uses ternary weights:

$$Q_W \in \{-1, 0, 1\}.$$

A common scale is based on the absolute mean:

$$s_W = \frac{1}{n} \sum_{i=1}^n |W_i|.$$

Then

$$W \approx s_W Q_W.$$

Activations can be quantized per token or per row to INT8. This makes it possible to use INT8 tensor cores even when weights are ternary, because the ternary values can be represented in an INT8-compatible format.

For distributed training, low-bit weights can reduce communication. In FSDP, weights are sharded across GPUs and all-gathered before a layer runs. If the all-gather communicates packed low-bit weights rather than BF16 weights, communication volume decreases.

Common Misconceptions

Quantization Is Only for Inference

Quantization can also target optimizer states, weights, activations, matrix multiplication inputs, and distributed communication buffers during training.

Low Precision Always Saves Memory

Low precision saves memory only if the low-precision representation is what is stored in global memory. If a full high-precision copy is materialized, the benefit is reduced or eliminated.

BF16 Cannot Have Underflow-Like Update Problems

BF16 has wide exponent range but limited mantissa precision. Small updates can still be lost when the spacing between representable BF16 values is too large.

INT8 Training Means INT8 Weights

INT8 training can mean using INT8 tensor cores while still keeping high-precision weights for the optimizer. This is often more practical than storing trainable weights only in INT8.

Tensorwise Scaling Is Enough

Tensorwise scaling is simple, but rowwise, columnwise, groupwise, and tilewise scaling can improve numerical accuracy.

Practical Takeaways

- Adam optimizer states are a major memory target because Adam keeps two state tensors per parameter.
- Low-bit optimizer states require fused dequantization, update, and requantization.
- Quantized trainable weights need stochastic rounding to avoid losing small updates.
- BF16 can also benefit from stochastic rounding when no FP32 master weights are kept.
- INT8 matrix multiplication can accelerate training when GEMMs dominate runtime.
- Rowwise and columnwise scaling are practical because the scales can be applied after the dot-product reduction.
- Inner-dimension scaling is harder because it enters the GEMM reduction loop.
- Triton is useful for writing fused kernels when PyTorch compilation does not generate the desired epilogue.
- Profiling is required to verify whether a quantization implementation actually saves memory or improves speed.

Project or Experiment Ideas

Experiment 1: Stochastic Rounding

Simulate repeated small updates with deterministic rounding and stochastic rounding. Verify that deterministic rounding stagnates while stochastic rounding preserves the expected value.

Experiment 2: BF16 Update Loss

Create BF16 tensors at different magnitudes and repeatedly add small values. Measure when updates stop changing the tensor. Then compare with stochastic rounding.

Experiment 3: Low-Bit Momentum Buffer

Implement SGD with FP32, BF16, FP8-simulated, and INT8-simulated momentum buffers. Compare loss curves and memory usage.

Experiment 4: Rowwise INT8 GEMM

Implement rowwise and columnwise scaled INT8 matrix multiplication. Compare it with tensorwise scaling using relative Frobenius error:

$$\epsilon = \frac{\|C_{\text{quant}} - C_{\text{fp}}\|_F}{\|C_{\text{fp}}\|_F}.$$

Experiment 5: Triton Fused Epilogue

Write a Triton GEMM kernel that applies row and column scales in the epilogue. Compare it against a separate scaling kernel.

Experiment 6: Tiny Transformer INT8 Training

Train a small transformer using BF16 GEMM and INT8 GEMM. Compare throughput, loss stability, and memory usage.

Final Mental Model

Quantized training is a set of targeted interventions, not a single recipe. The full training step is

$$\text{training} = \text{forward} + \text{backward} + \text{optimizer} + \text{communication}.$$

Each part has a different quantization opportunity:

optimizer states $\rightarrow$ memory savings,

trainable weights $\rightarrow$ memory savings but update-risk,

matmul inputs $\rightarrow$ tensor-core speed,

activations $\rightarrow$ bandwidth savings,

communication $\rightarrow$ distributed bandwidth savings.

The correct engineering rule is:

Use low precision where it reduces HBM traffic or unlocks faster tensor cores. Use higher precision where long-term numerical state must remain stable. Use stochastic rounding when low precision would otherwise erase small updates.

Visual Concept

Draw a training memory timeline. Parameters and optimizer states persist across the step. Activations grow during forward and shrink during backward. Gradients grow during backward and disappear during the optimizer step. Mark each region as a quantization opportunity.

Visual Concept

Draw a fused low-bit optimizer kernel. On the left, show quantized optimizer states in HBM. Inside the SM, show dequantization into registers or shared memory, optimizer math, scale computation, and requantization. On the right, show only quantized states written back to HBM.

Visual Concept

Draw rowwise and columnwise scaled GEMM. Matrix A has one scale per row. Matrix B has one scale per column. Each output element C_{ij} receives the product $s_i^A s_j^B$ after the integer dot product.

Visual Concept

Draw stochastic rounding on a number line. Place x between $\lfloor x \rfloor$ and $\lfloor x \rfloor + 1$. Show that the probability of rounding upward equals the fractional distance from the lower value.